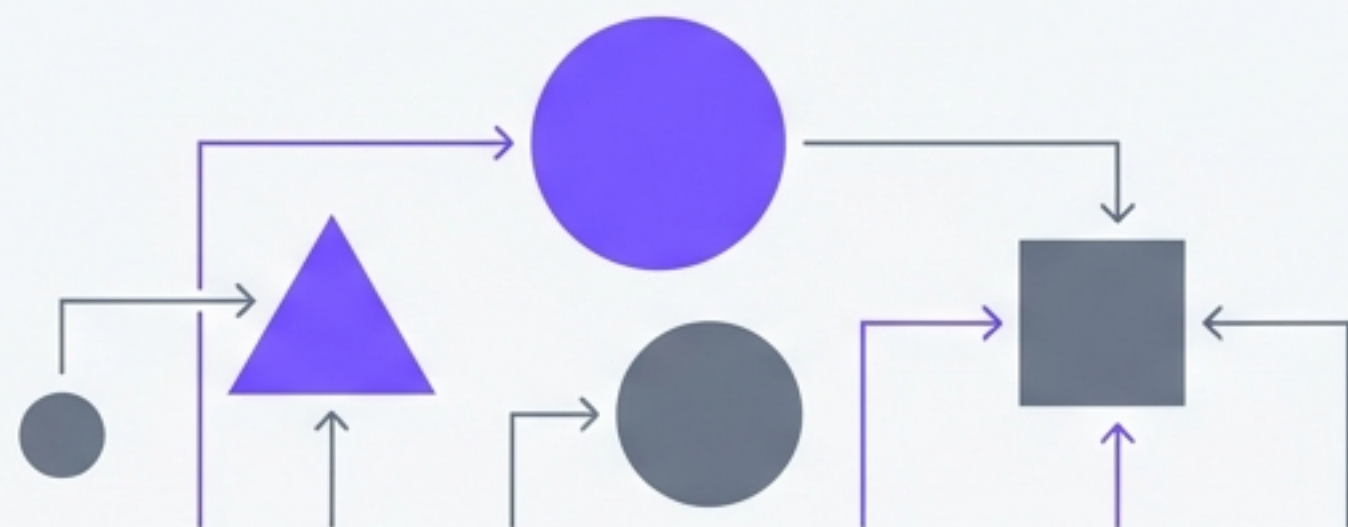
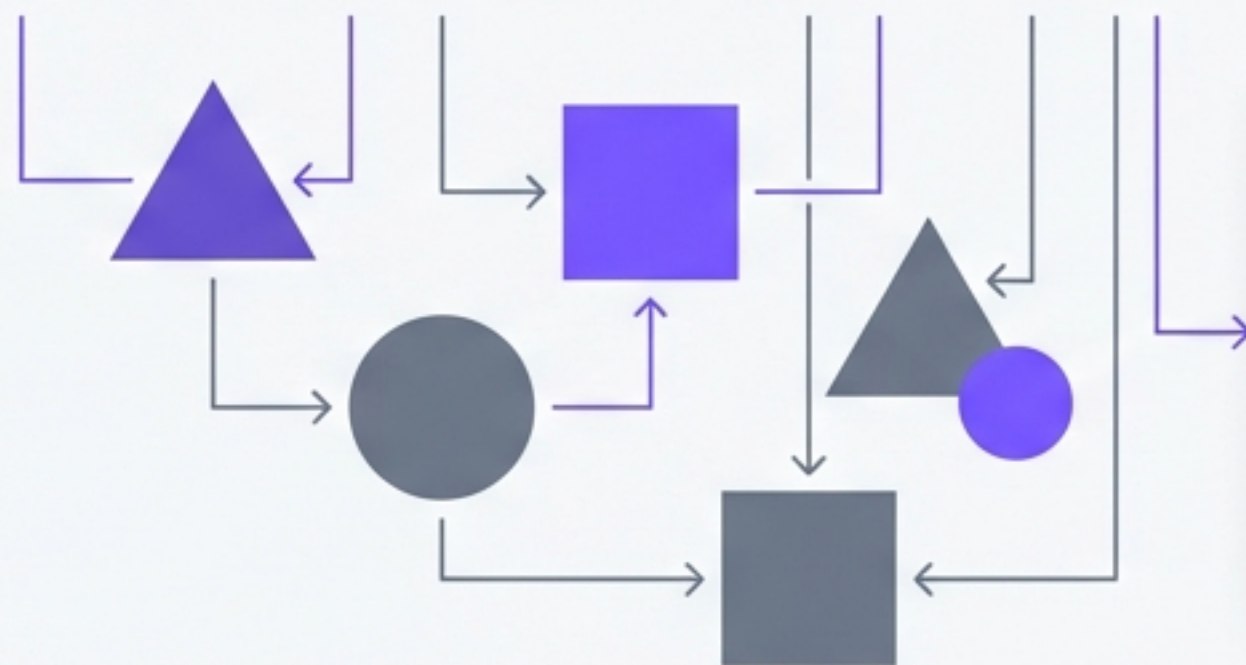




Kotlin: Basic Syntax Overview

A visual guide to the modern developer's toolkit.



```
fun main() {  
    println("Hello, World!")  
  
    val name: String = "Kotlin"  
    var version: Double = 1.9  
    version = 2.0  
    println("Welcome to $name version $version")  
}
```

Based on official Kotlin Documentation and JetBrains Academy Core track.

Program Structure & Entry Points

Anatomy of a File

Top Layer

```
package my.demo
```

Package definition
(independent of
directory structure)

Imports

```
import kotlin.text.*
```

Entry Point

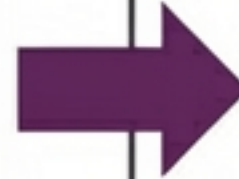
```
fun main() {  
    println("Hello world!")  
}
```

```
fun main(args: Array<String>) { ... }
```


Input, Output, and String Templates

CODE (Input)

```
println("Enter word:")  
val word = readln()  
print("You wrote: \u001b[36m$word\u001b[0m")
```

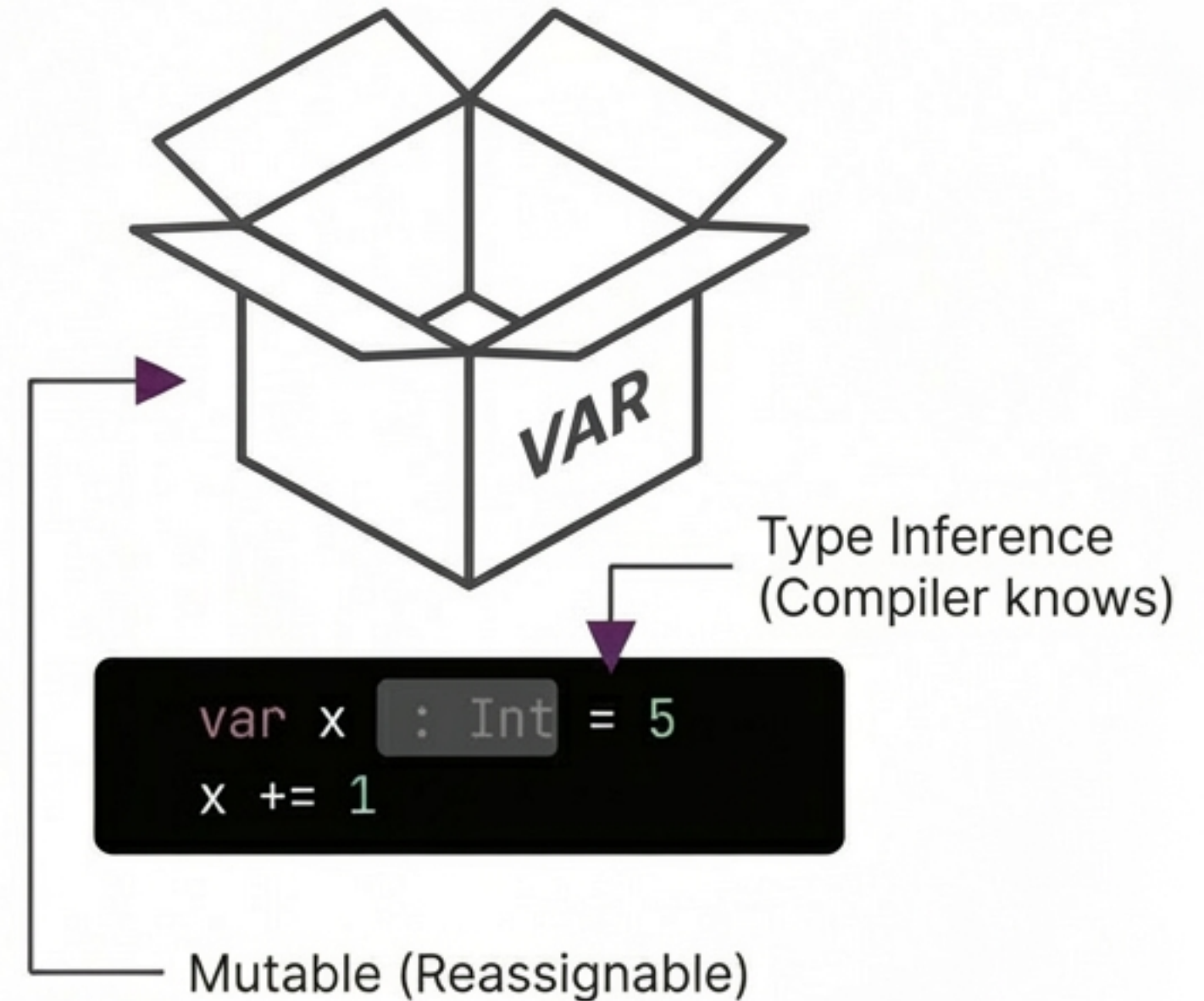
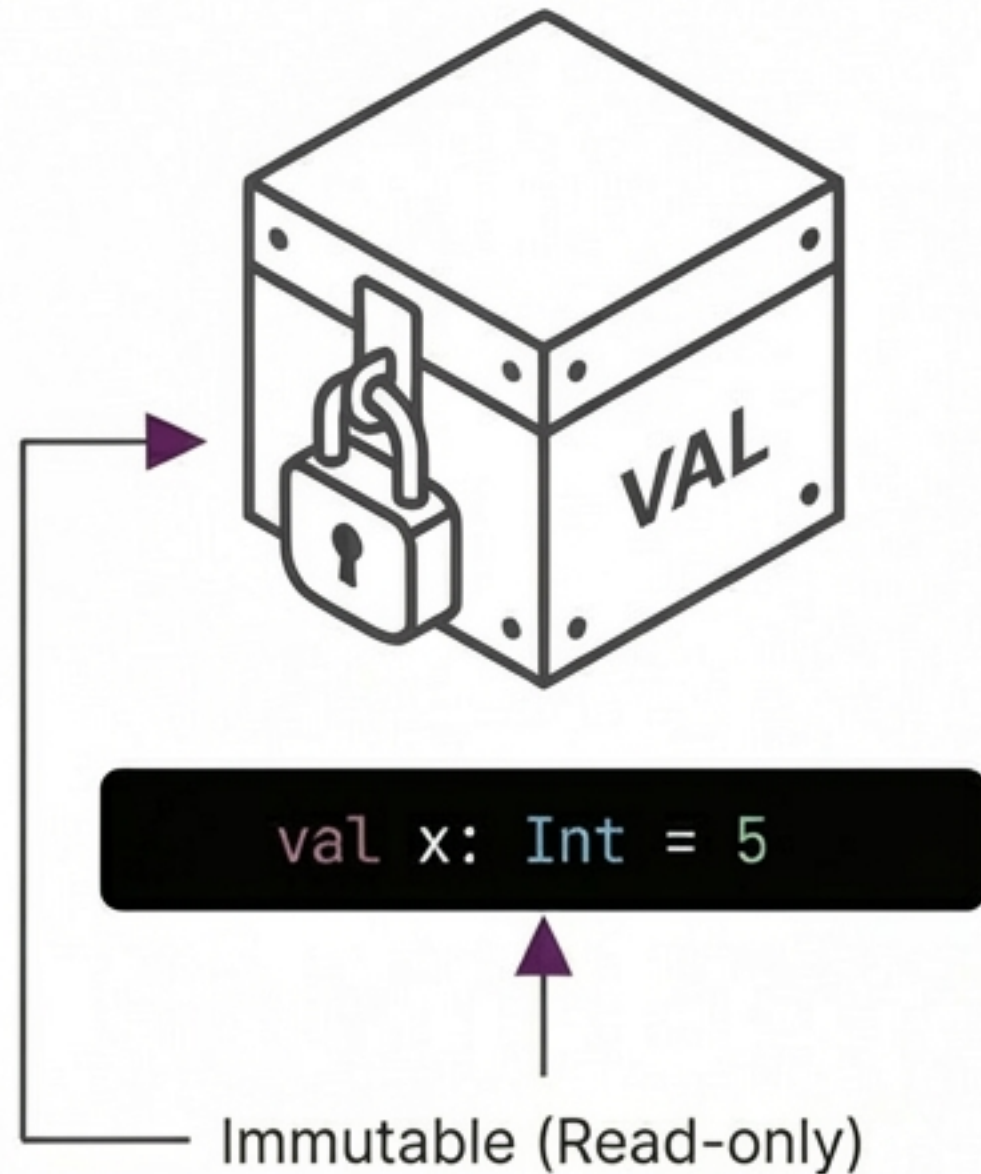


CONSOLE (Output)

```
> Enter word:  
> Happiness  
> You wrote: Happiness
```

- println vs print:
New line vs continuous.
- readln():
Captures standard input.
- Templates:
Insert variables directly with \$

Variables & Type Inference

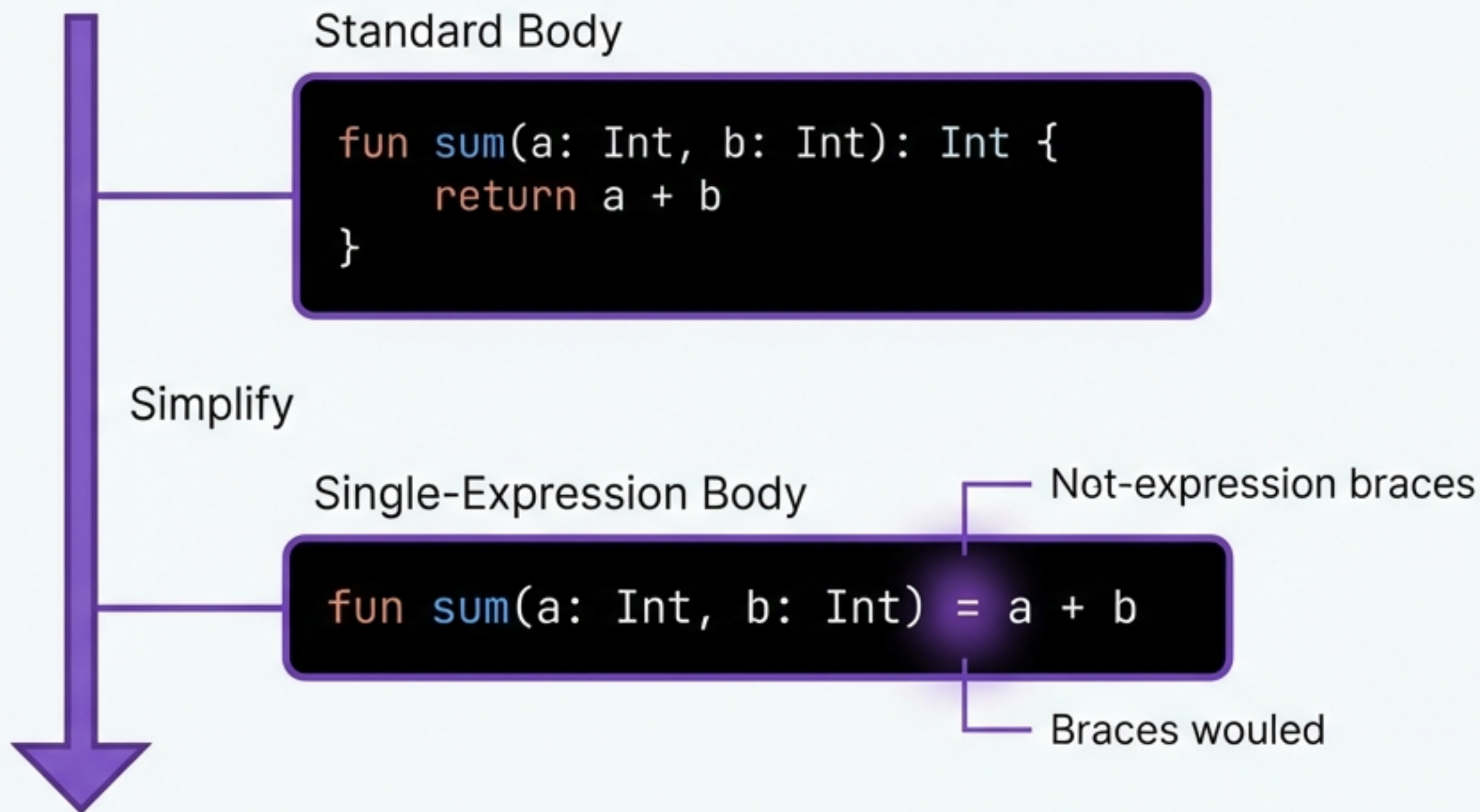


Advanced

```
val PI = 3.14 // Top-level declaration
```


Functions: The Workhorses

Evolution



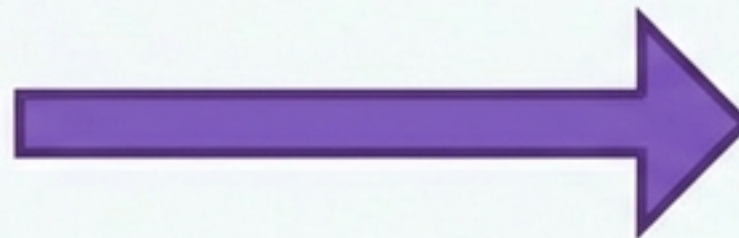
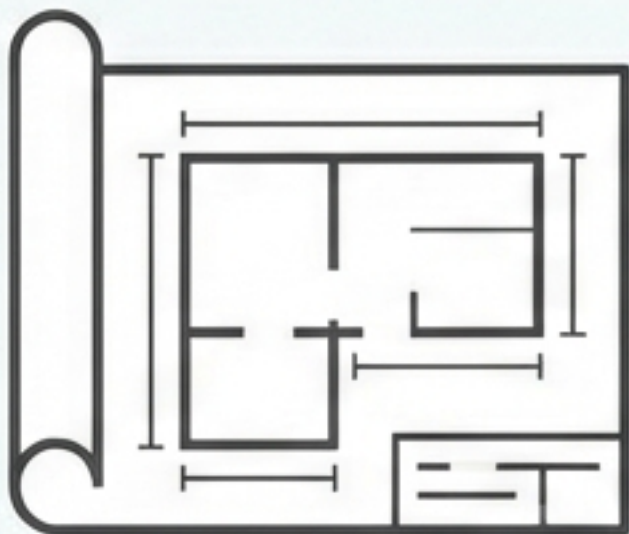
The Void Type

```
fun printSum(...): Unit
```

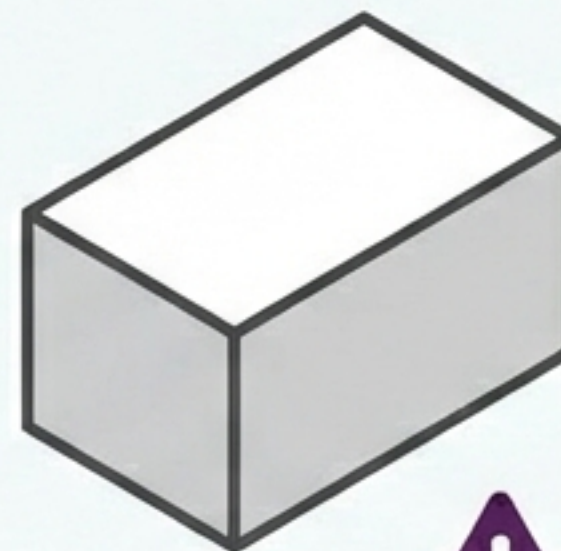
Unit = No meaningful return value (optional to write).

Classes & Instances

Class Definition



Instance

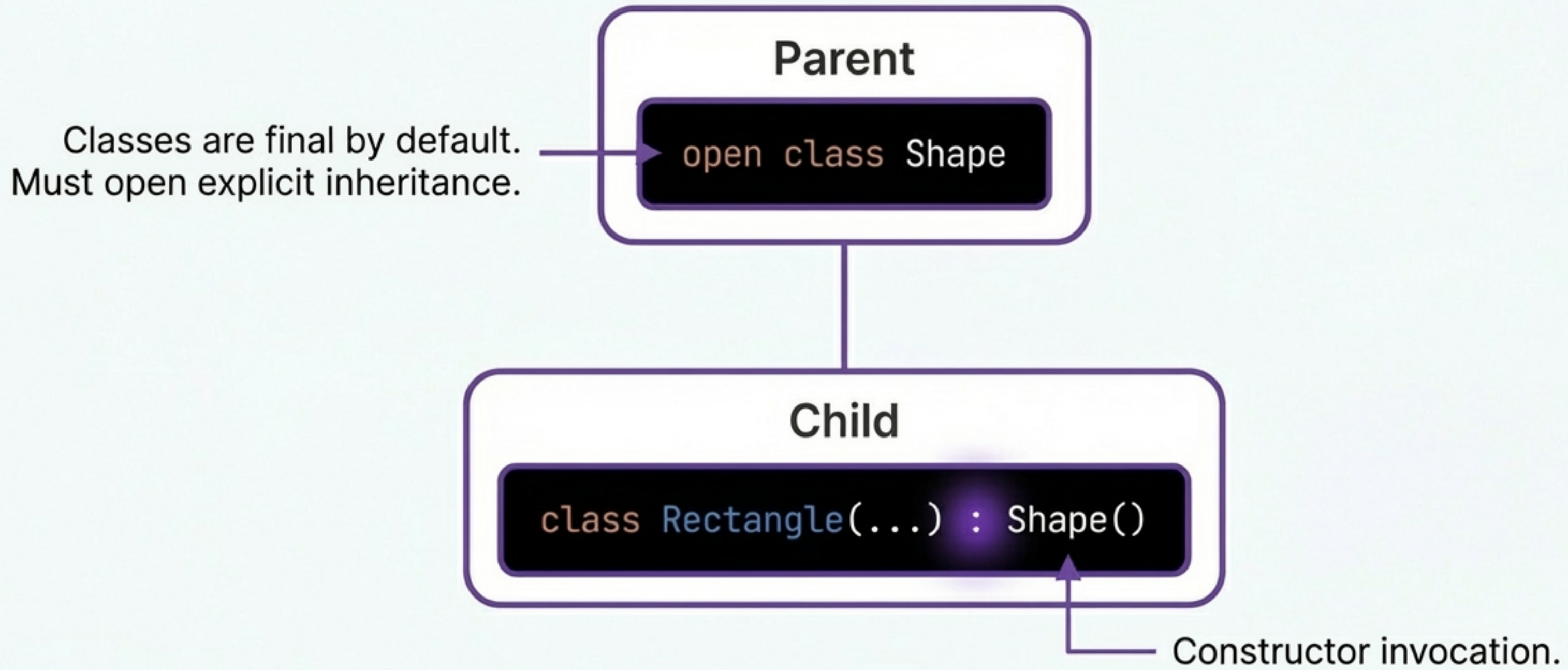


No 'new' keyword required.

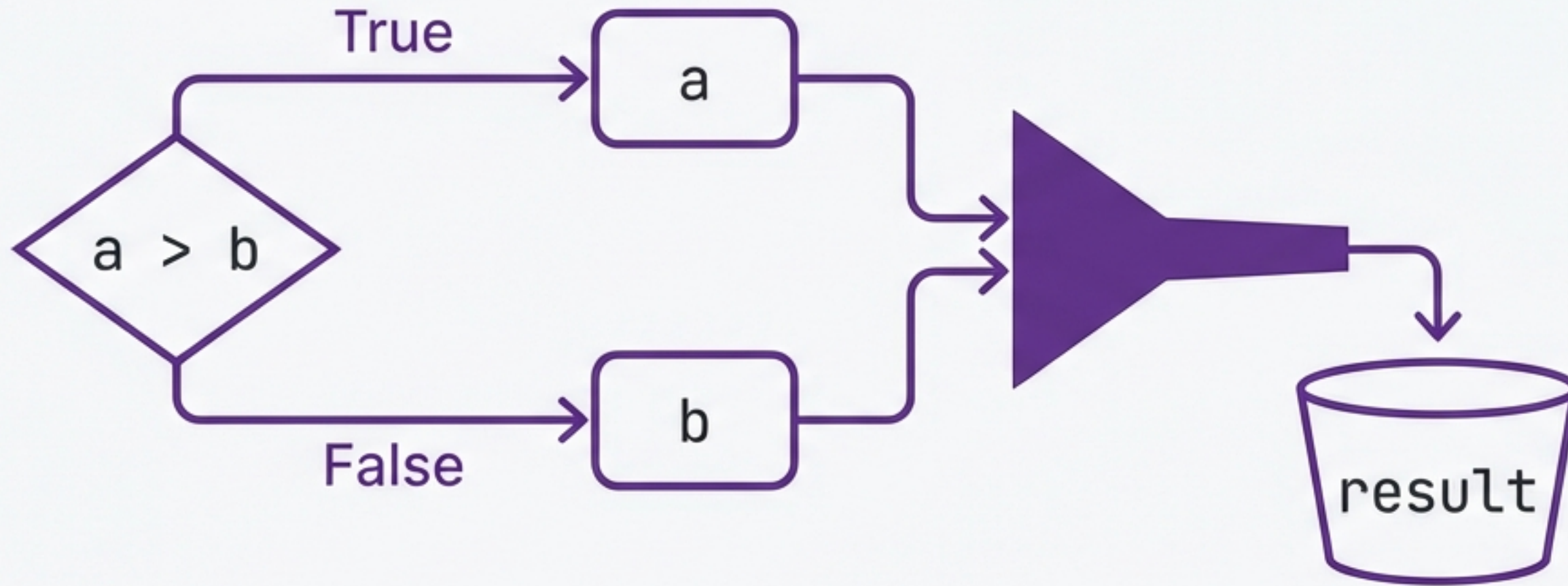
```
class Rectangle(val height: Double,  
                val length: Double) {  
    val perimeter = (height + length) * 2  
}
```

```
val rectangle = Rectangle(5.0, 2.0)  
println(rectangle.perimeter)
```


Inheritance Hierarchy



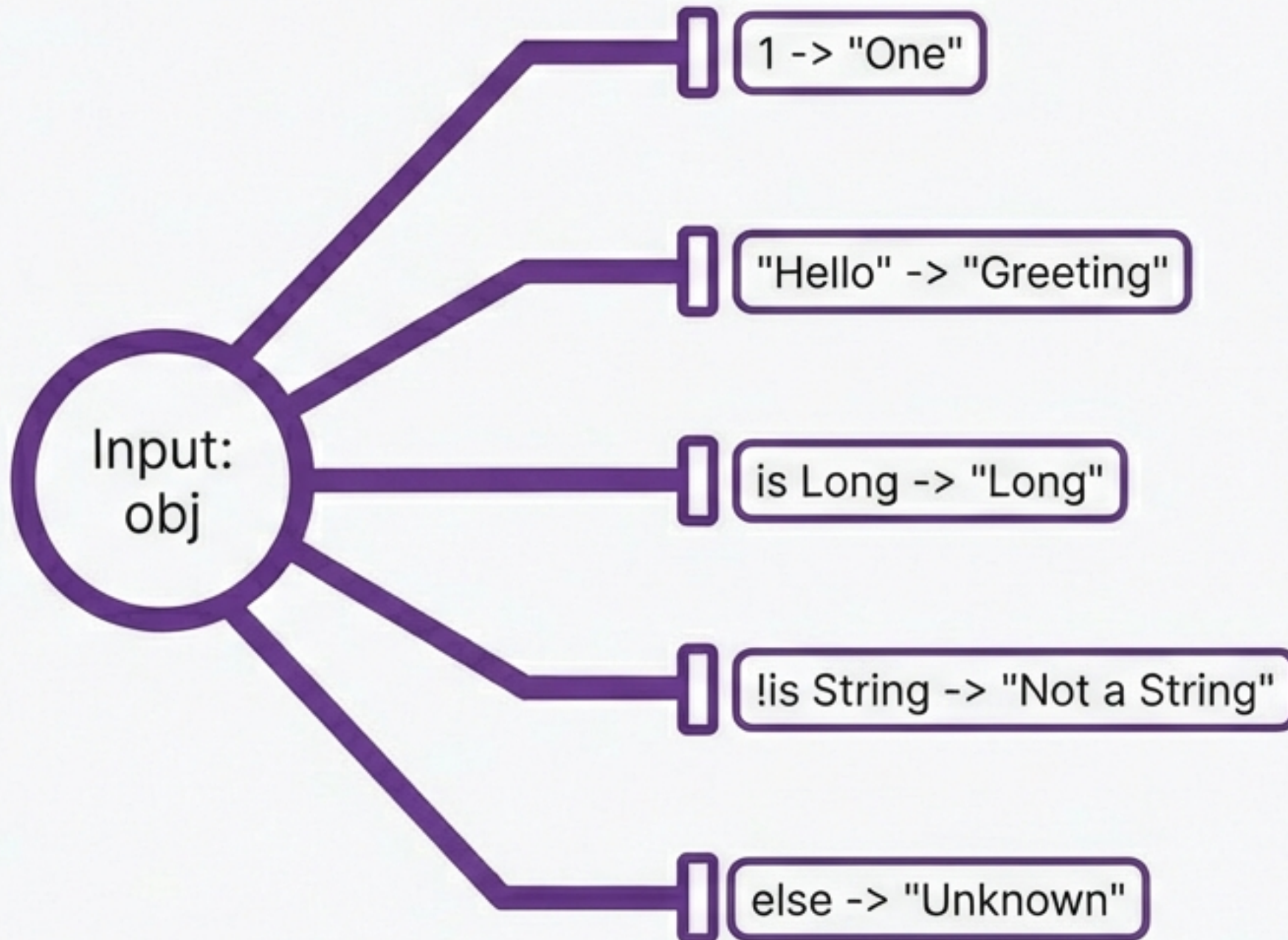
Conditional Expressions



```
fun maxOf(a: Int, b: Int) = if (a > b) a else b
```

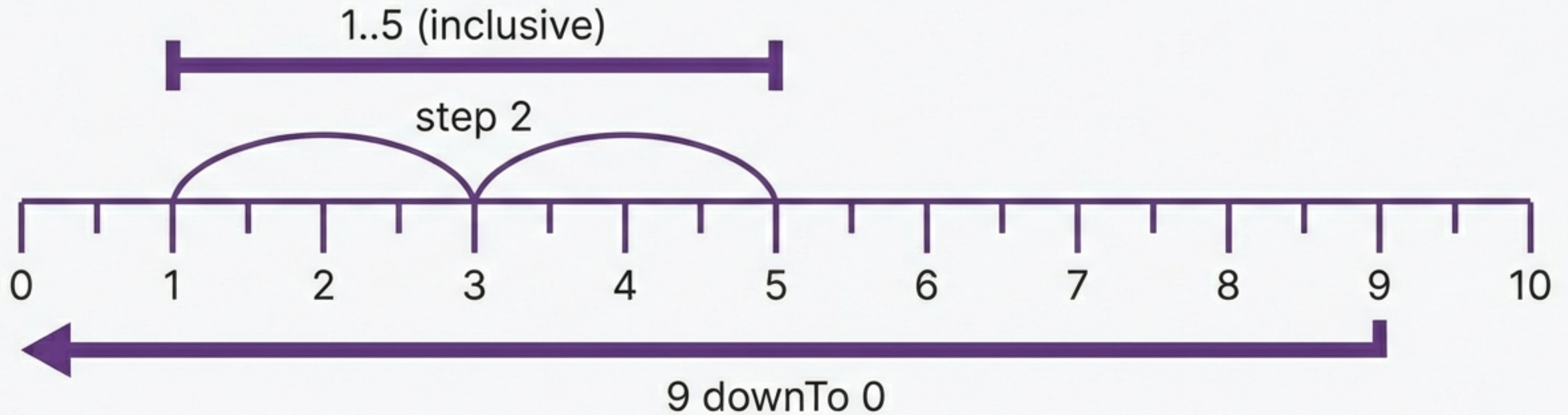
In Kotlin, 'if' returns a value.

The 'when' Expression



```
when (obj) {  
  1 -> "One"  
  is Long -> "Long"  
  else -> "Unknown"  
}
```


Loops & Ranges

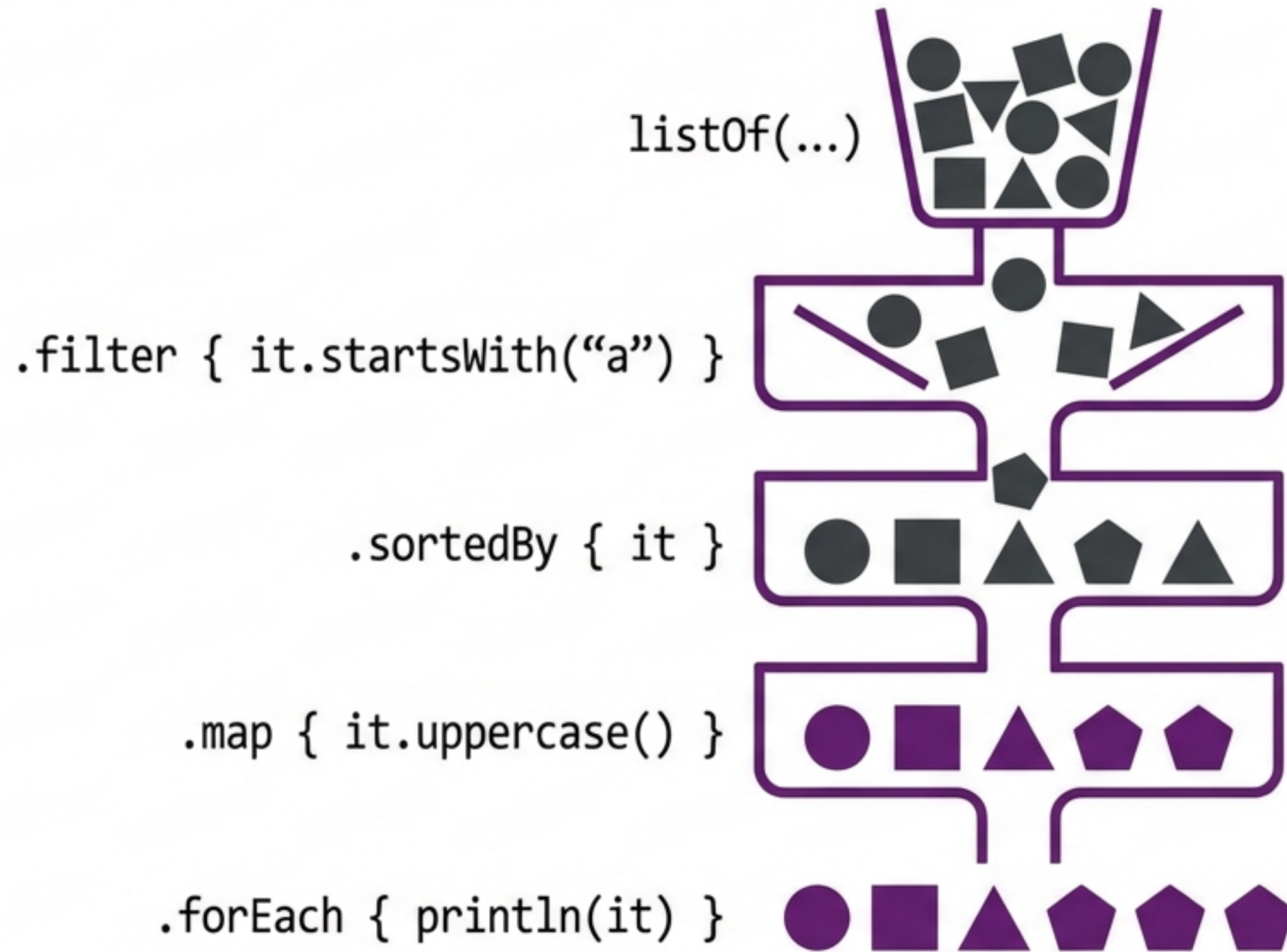


```
for (x in 1..5) {  
    ...  
}
```

```
for (x in 1..10 step 2) {  
    ...  
}
```

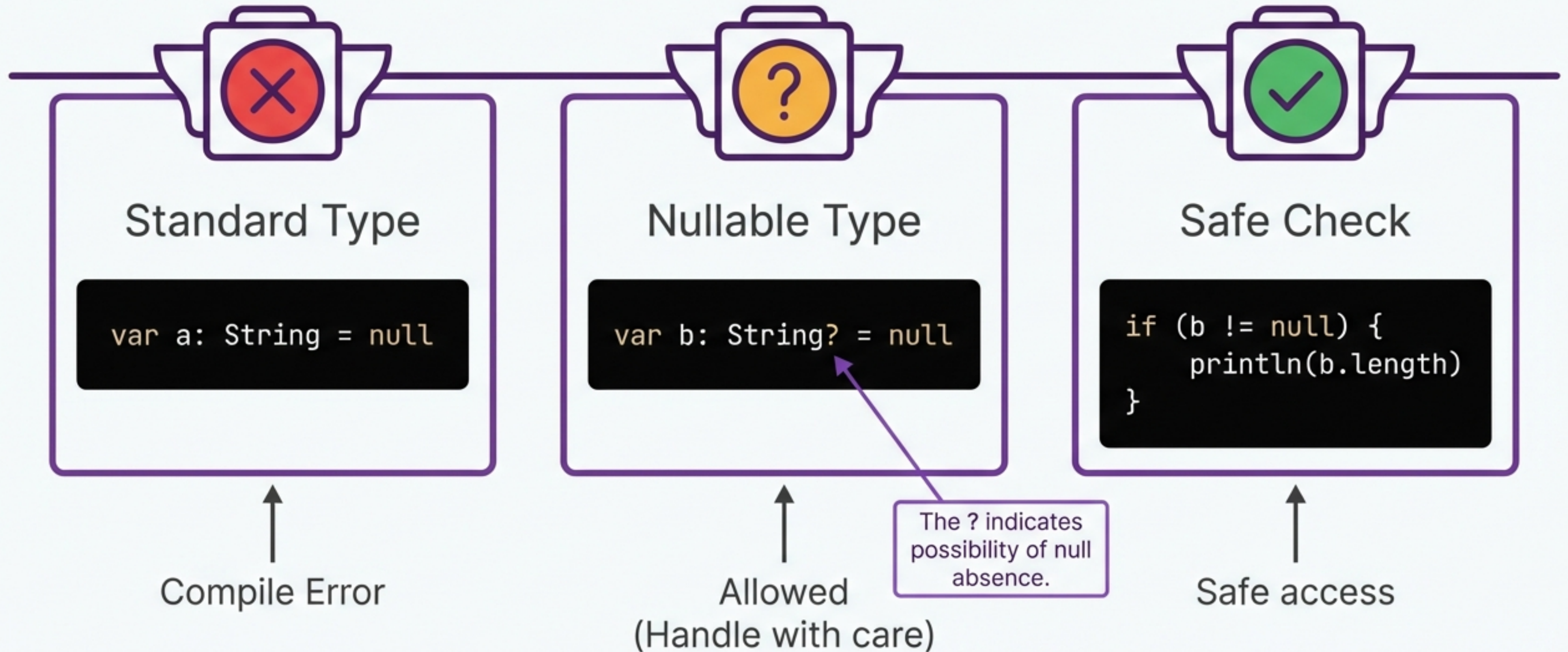
```
if (x in 1..y) {  
    ...  
} // Bounds check
```


Collections & Functional Lambdas

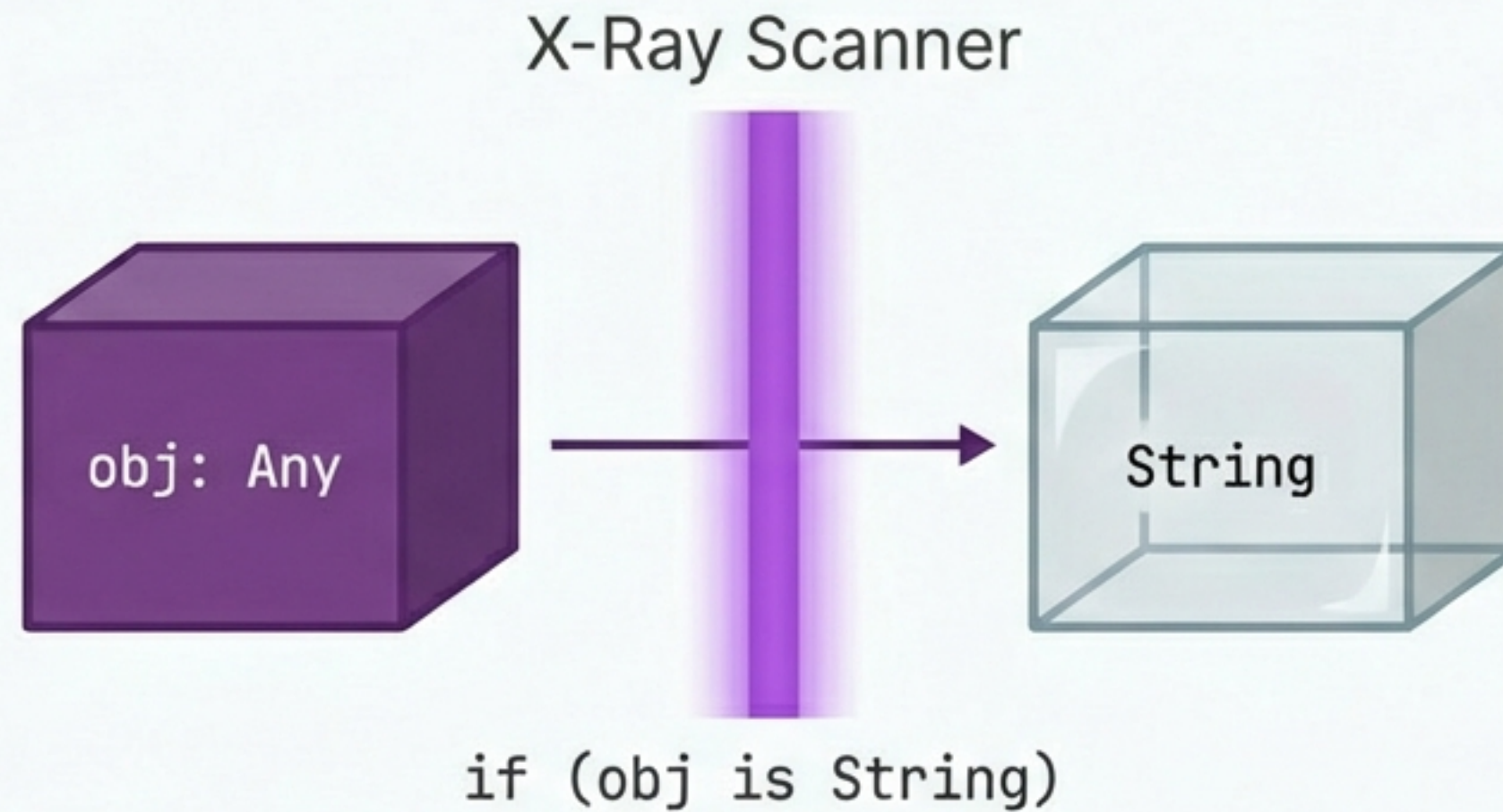


'it' = implicit name of a single parameter.

Null Safety: The Shield



Type Checks & Smart Casts



```
if (obj is String) {  
    // obj is automatically cast to String here  
    return obj.length  
}
```

No explicit
casting needed.

The Modern Toolkit Recap

Essentials

- Entry Points (`main`)
- `val` (immutable) vs `var`
- String Templates (`$`)

Building Blocks

- Single-Expression Functions
 - Primary Constructors
 - Explicit Inheritance

Flow Control

- `if/when` as expressions
- Ranges (`1..5`)
- Rich loops

Safety

- Nullable types (`?`)
- Smart Casts (`is`)
- Functional Pipelines

Read the documentation at kotlinlang.org | Practice at JetBrains Academy.